



เอกสารสรุปผลการพัฒนา — Fall Guard Web App

เครื่องประมวลผลภาพอัจฉริยะเพื่อตรวจจับและแจ้งเตือนการล้มของผู้สูงอายุภายในบ้านด้วยปัญญาประดิษฐ์ (AI) ผ่านระบบไลน์

สารบัญ



เอกสารสรุปผลการพัฒนา — Fall Guard Web App

การอัปเดตล่าสุด (สิงหาคม 2569): ปรับปรุง Desktop App และเพิ่มฟีเจอร์ AI

- ภาพรวมสิ่งที่ทำ
- สถาปัตยกรรมระบบ
- โครงสร้างไฟล์ที่สร้าง/แก้ไข
- โครงสร้างฐานข้อมูล
- API Endpoints ทั้งหมด
- ระบบ Authentication
- ระบบ Pairing (จับคู่อุปกรณ์)
- ระบบ Live Camera Feed
- ระบบ Fall Event Logging
- สิ่งที่ต้อง Config เพิ่มเติม
- วิธีรันระบบ
- ข้อจำกัดและแนวทางพัฒนาต่อ

การอัปเดตล่าสุด (สิงหาคม 2569): ปรับปรุง Desktop App และเพิ่มฟีเจอร์ AI

การพัฒนาก่อนหน้านี้เน้นไปที่ Web App แต่การอัปเดตล่าสุดได้มุ่งเน้นไปที่การปรับปรุง Core System (Desktop App) และเพิ่มความสามารถของ AI ดังนี้:

- สถาปัตยกรรม MVC (Model-View-Controller)
 - ปรับโครงสร้างโค้ดของ Desktop App (Flet) ทั้งหมดให้อยู่ในรูปแบบ MVC (src/mvc/)

- แยกส่วนการจัดการกล้อง (Camera Manager), การสตรีมวิดีโอ, และ UI ออกจากกันอย่างชัดเจน เพื่อให้โค้ดเป็นระเบียบและง่ายต่อการบำรุงรักษา
2. การอัปเดต AI Detection (Multipose & Intruder)
- อัปเดตไปใช้โมเดล **Multipose** และปรับปรุงลอจิกการตรวจจับการล้มให้รองรับ multi-class pose labels เพื่อความแม่นยำที่สูงขึ้น
 - เพิ่มระบบ **Face Registration (การลงทะเบียนใบหน้า)** ผ่านเว็บแคม (src/services/face_recognition_service.py)
 - เพิ่มระบบ **Intruder Detection (ตรวจจับผู้บุกรุก)** พร้อมระบบบันทึกเหตุการณ์ (Logging) และ Event Callbacks
3. การแจ้งเตือนผ่าน LINE
- พัฒนาการแจ้ง ผ่าน LINE เข้ากับระบบ เมื่อเกิดเหตุการณ์ฉุกเฉิน (การล้ม หรือตรวจพบผู้บุกรุก) ระบบจะส่งข้อความแจ้งเตือนได้ทันที
4. การค้นหาและจัดการกล้องผ่านเครือข่าย (Network Discovery)
- ปรับปรุงระบบค้นหาและจัดการกล้อง IP Camera (ONVIF) ให้ทำงานร่วมกับโครงสร้าง MVC ใหม่ได้อย่างสมบูรณ์

1. ภาพรวมสิ่งที่ทำ

สร้าง Web Application แยกต่างหาก สำหรับดูภาพกล้องวงจรปิดที่มีระบบตรวจจับการล้มผ่านโทรศัพท์มือถือ โดยมีฟีเจอร์หลัก 4 อย่าง:

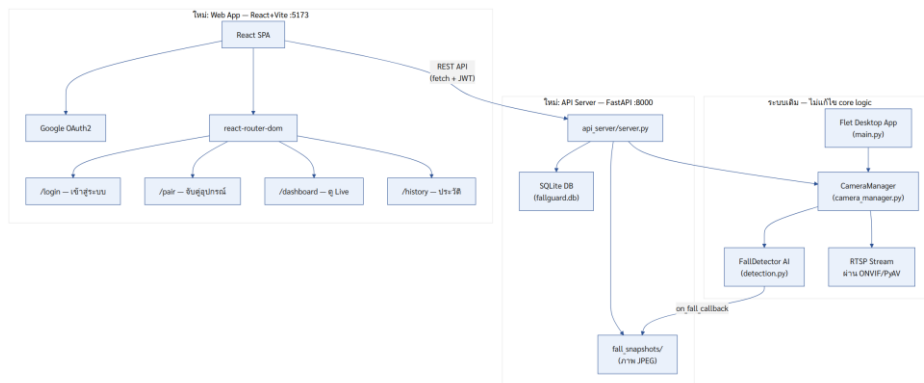
ฟีเจอร์	รายละเอียด
 Google Login	ผู้ใช้งานลงทะเบียน/เข้าสู่ระบบด้วยบัญชี Google (มี Dev Mode สำหรับทดสอบ)
 จับคู่อุปกรณ์	กรอกรหัส 6 หลักเพื่อเชื่อมต่อกับระบบ detection ที่รันอยู่บนคอมพิวเตอร์
 ดูภาพ Live	ดูภาพกล้องวงจรปิดแบบ real-time ผ่านเว็บบนมือถือ (polling ทุก 1.5 วินาที)
 ประวัติการล้ม	ดูเหตุการณ์ล้มย้อนหลังพร้อมภาพ snapshot ขณะตรวจพบ

เทคโนโลยีที่ใช้

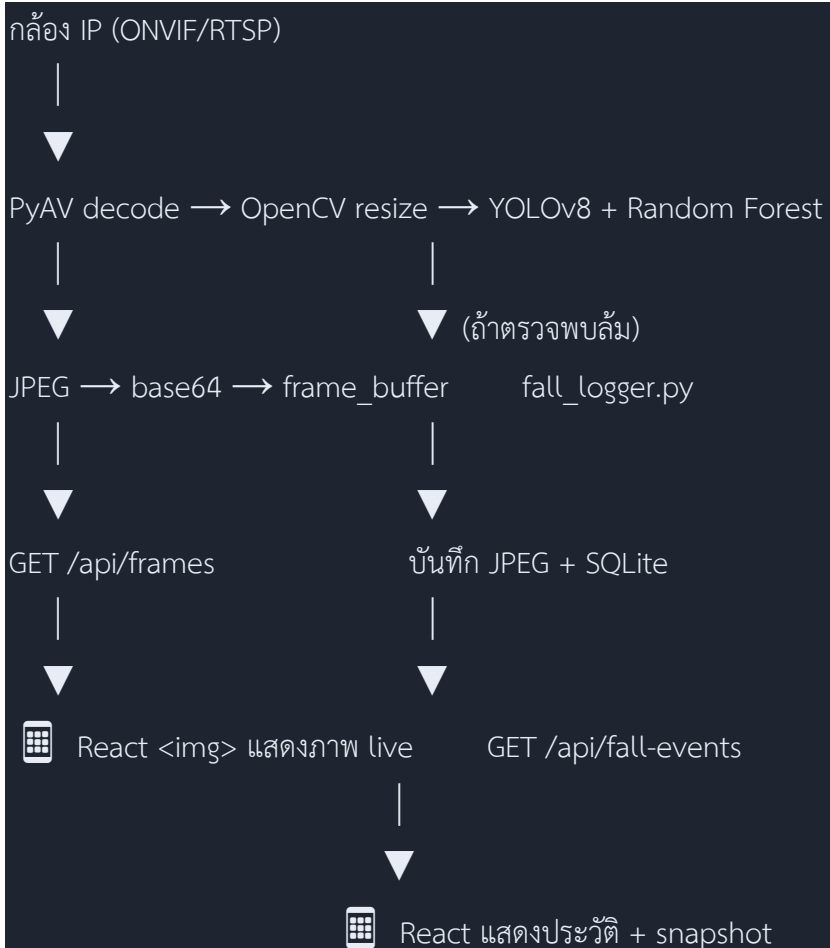
ส่วน	เทคโนโลยี
Frontend	React 19, Vite, react-router-dom, framer-motion, @react-oauth/google

ส่วน	เทคโนโลยี
Backend	FastAPI, uvicorn, SQLite (WAL mode), PyJWT, google-auth
ระบบเดิม	Flet, YOLOv8, scikit-learn, PyAV, OpenCV, onvif-zeep
Design	CSS Custom Properties, Glassmorphism, Dark Theme "Midnight Luxury"

2. สถาปัตยกรรมระบบ



การไหลของข้อมูลหลัก



3. โครงสร้างไฟล์ที่สร้าง/แก้ไข

☒ ไฟล์ที่สร้างใหม่ (27 ไฟล์)

```
d:\buff_p\  
|  
|  
|—— SKILL.md          ← เอกสาร context สำหรับ AI agents  
|—— .env              ← ตัวแปรสำหรับ API server  
|—— system_config.json ← System ID (สร้างอัตโนมัติ)  
|—— fallguard.db       ← ฐานข้อมูล SQLite (สร้างอัตโนมัติ)  
|—— fall_snapshots/    ← โฟลเดอร์ภาพ snapshot (สร้างอัตโนมัติ)  
|  
|—— api_server/        ← ★ API Server ทั้งหมด  
|   |—— __init__.py  
|   |—— server.py      ← FastAPI app + ทุก endpoint  
|   |—— database.py    ← SQLite schema + CRUD functions  
|   |—— auth.py        ← Google OAuth2 verify + JWT  
|   |—— models.py      ← Pydantic request/response schemas  
|   |—— pairing.py     ← รหัสจับคู่ 6 หลัก + System ID  
|   |—— fall_logger.py ← บันทึกเหตุการณ์ล้ม + snapshot  
|  
|—— web_app/          ← ★ React Web App ทั้งหมด  
|   |—— .env          ← ตัวแปรสำหรับ frontend  
|   |—— vite.config.js ← Vite config + API proxy  
|   |—— index.html     ← HTML entry (Inter font, meta tags)  
|   |—— public/  
|       |—— favicon.svg ← โลโก้ Fall Guard  
|   |—— src/  
|       |—— main.jsx   ← Entry: providers + router  
|       |—— App.jsx    ← Routes + bottom nav
```

```

├── index.css          ← Design System ทั้งหมด (280 บรรทัด)
├── pages/
│   ├── LoginPage.jsx + .css ← หน้าเข้าสู่ระบบ
│   ├── PairPage.jsx + .css  ← หน้าจับคู่อุปกรณ์
│   ├── DashboardPage.jsx + .css ← หน้าดูภาพ live
│   └── HistoryPage.jsx + .css ← หน้าประวัติการล้ม
├── components/
│   ├── BottomNav.jsx + .css ← แถบเมนูด้านล่าง
│   ├── CameraCard.jsx + .css ← การ์ดแสดงภาพกล้อง
│   ├── FallEventCard.jsx + .css ← การ์ดเหตุการณ์ล้ม
│   ├── LiveBadge.jsx + .css ← ป้าย "● LIVE"
│   ├── StatusChip.jsx + .css ← ป้ายสถานะ ปกติ/ล้ม
│   ├── PairCodeInput.jsx + .css ← ช่องกรอกรหัส 6 หลัก
│   └── ProtectedRoute.jsx ← ตัวป้องกันหน้าที่ต้อง login
├── hooks/
│   └── useCameraFeed.js ← Hook ดึงภาพกล้อง (polling)
├── lib/
│   └── api.js ← Fetch wrapper + auth header
├── contexts/
│   └── AuthContext.jsx ← React Context สำหรับ auth state

```

ไฟล์ที่แก้ไข (2 ไฟล์)

ไฟล์	สิ่งที่แก้ไข
detection.py	เพิ่ม parameter on_fall_callback ใน FallDetector.__init__() และเรียก callback เมื่อตรวจพบการล้ม (ไม่แก้ AI logic เดิม)
src/services/playvideo.py	เพิ่ม function _on_fall() ที่เรียก fall_logger.log_fall_event() เมื่อตรวจพบการล้ม

4. โครงสร้างฐานข้อมูล

ใช้ **SQLite** ในโหมด **WAL** (Write-Ahead Logging) สำหรับ concurrent reads ที่ดีขึ้น

ไฟล์ฐานข้อมูล: d:\buff_p\fallguard.db

ตาราง users — ข้อมูลผู้ใช้งาน

คอลัมน์	ชนิด	คำอธิบาย
id	INTEGER PRIMARY KEY	รหัสผู้ใช้ (auto increment)
google_id	TEXT UNIQUE NOT NULL	Google Account ID (จาก OAuth)
email	TEXT NOT NULL	อีเมล
name	TEXT	ชื่อที่แสดง
picture	TEXT	URL รูปโปรไฟล์
created_at	REAL	Timestamp สร้างบัญชี (Unix)

ตาราง pairings — การจับคู่ผู้ใช้กับระบบ

คอลัมน์	ชนิด	คำอธิบาย
id	INTEGER PRIMARY KEY	รหัสการจับคู่
user_id	INTEGER FK → users(id)	ผู้ใช้ที่จับคู่
system_id	TEXT NOT NULL	UUID ของระบบ detection (เช่น b475ed54)
pair_code	TEXT	รหัส 6 หลักที่ใช้จับคู่
is_active	INTEGER (0/1)	สถานะ: 0=ยกเลิกแล้ว, 1=ใช้งานอยู่
created_at	REAL	Timestamp สร้างรหัส
paired_at	REAL	Timestamp ที่จับคู่สำเร็จ
expires_at	REAL	Timestamp หมดอายุ

ตาราง fall_events — เหตุการณ์การล้ม

คอลัมน์	ชนิด	คำอธิบาย
id	INTEGER PRIMARY KEY	รหัสเหตุการณ์
camera_ip	TEXT NOT NULL	IP ของกล้องที่ตรวจพบ

คอลัมน์	ชนิด	คำอธิบาย
camera_name	TEXT	ชื่อก້ອງ (เช่น "ห้องพักครู")
snapshot_filename	TEXT	ชื่อไฟล์ภาพ snapshot (เช่น 20260602_143215_192_168_10_55.jpg)
detected_at	REAL NOT NULL	Timestamp ที่ตรวจพบ (Unix)
duration_seconds	REAL	ระยะเวลาที่ตรวจพบ (วินาที)
created_at	REAL	Timestamp บันทึกลง DB

Indexes

```
CREATE INDEX idx_fall_events_detected_at ON fall_events(detected_at);
CREATE INDEX idx_pairings_user_id ON pairings(user_id);
CREATE INDEX idx_pairings_pair_code ON pairings(pair_code);
```

หมายเหตุเรื่องการจัดเก็บภาพ

ภาพ snapshot ไม่ได้เก็บในฐานข้อมูล — เก็บเป็นไฟล์ .jpg ในโฟลเดอร์ fall_snapshots/ ในฐานข้อมูลเก็บแค่ ชื่อไฟล์ (snapshot_filename) เท่านั้น

เมื่อ Web App ต้องการแสดงภาพ จะเรียกผ่าน GET /api/snapshots/{filename} ซึ่ง FastAPI จะ serve ไฟล์ภาพจริงกลับมา

5. API Endpoints ทั้งหมด

API Server รันที่ http://0.0.0.0:8000

5.1 Authentication (ไม่ต้อง login)

POST /api/auth/google — เข้าสู่ระบบด้วย Google

Request Body:

```
{
  "credential": "eyJhbGciOiJSUzI1NiIs..."
}
```

- credential — Google ID Token ที่ได้จาก @react-oauth/google
- ถ้าไม่ได้ตั้ง GOOGLE_CLIENT_ID (Dev Mode) จะสร้าง test user ให้อัตโนมัติ

Response (200):

```
{
  "token": "eyJhbGciOiJIUzI1NiIs... ",
  "user": {
    "id": 1,
    "google_id": "118234567890",
    "email": "user@gmail.com",
    "name": "สมชาย",
    "picture": "https://lh3.googleusercontent.com/..."
  }
}
```

GET /api/auth/me — ดูข้อมูลผู้ใช้ปัจจุบัน 

Header: Authorization: Bearer <JWT token>

Response (200):

```
{
  "id": 1,
  "google_id": "118234567890",
  "email": "user@gmail.com",
  "name": "สมชาย",
  "picture": "https://..."
}
```

5.2 Pairing (จับคู่อุปกรณ์)

GET /api/pair/generate — สร้างรหัสจับคู่ใหม่

เรียกจากฝั่ง server/admin — แสดงรหัสบนหน้าจอ Flet desktop app

Response (200):

```
{
  "code": "483921",
  "expires_in": 600,
}
```



```
"system_id": "b475ed54"
}
```

GET /api/pair/current — ดูรหัสจับคู่ปัจจุบัน

ถ้าไม่มีรหัสที่ยัง valid จะสร้างใหม่ให้อัตโนมัติ

Response: เหมือน */api/pair/generate*

POST /api/pair — จับคู่ด้วยรหัส 6 หลัก 

Request Body:

```
{
  "code": "483921"
}
```

Response สำเร็จ (200):

```
{
  "success": true,
  "message": "จับคู่สำเร็จ!",
  "system_id": "b475ed54"
}
```

Response ล้มเหลว (400):

```
{
  "detail": "รหัสไม่ถูกต้องหรือหมดอายุแล้ว"
}
```

GET /api/pair/status — เช็คสถานะการจับคู่ 

Response (จับคู่แล้ว):

```
{
  "is_paired": true,
  "system_id": "b475ed54",
  "paired_at": 1748859300.5
}
```

Response (ยังไม่จับคู่):

```
{
  "is_paired": false,
  "system_id": null,
  "paired_at": null
}
```

DELETE /api/pair — ยกเลิกการจับคู่ 

Response (200):

```
{
  "success": true,
  "message": "ยกเลิกการจับคู่แล้ว"
}
```

5.3 กล้อง (Cameras)

GET /api/cameras — รายชื่อกล้องที่ออนไลน์ 

Response (200):

```
[
  {
    "ip": "192.168.10.55",
    "name": "ห้องพัสดุ",
    "has_frame": true
  },
  {
    "ip": "192.168.10.60",
    "name": "ห้องนอน",
    "has_frame": true
  }
]
```

GET /api/frames — ดึงภาพ base64 ทุกกล้อง 

Response (200):

```
{
  "192.168.10.55": "/9j/4AAQSkZJRg...(base64 JPEG)...",
  "192.168.10.60": "/9j/4AAQSkZJRg..."
}
```

Web App จะเรียก endpoint นี้ทุก 1.5 วินาที แล้วแสดงเป็น ``

GET /api/frames/{ip} — ดึงภาพกล้องตัวเดียว 

ตัวอย่าง: `GET /api/frames/192.168.10.55`

Response (200):

```
{
  "ip": "192.168.10.55",
  "frame": "/9j/4AAQSkZJRg..."
}
```

5.4 เหตุการณ์การล่ม (Fall Events)

GET /api/fall-events — ดูประวัติการล่ม 

Query Parameters:

Parameter	ชนิด	ค่าเริ่มต้น	คำอธิบาย
limit	int	50	จำนวนรายการสูงสุด
offset	int	0	เริ่มจากรายการที่
camera	string	—	กรองตาม IP กล้อง
date_from	string	—	วันเริ่มต้น (YYYY-MM-DD)
date_to	string	—	วันสิ้นสุด (YYYY-MM-DD)

ตัวอย่าง: `GET /api/fall-events?date_from=2026-06-02&limit=10`

Response (200):

```
{
  "events": [
    {
      "id": 1,
      "camera_ip": "192.168.10.55",
      "camera_name": "ห้องพักครู",
      "snapshot_filename": "20260602_143215_192_168_10_55.jpg",
      "snapshot_url": "/api/snapshots/20260602_143215_192_168_10_55.jpg",
      "detected_at": 1748859135.0,
      "detected_at_formatted": "02/06/2026 14:32:15",
      "duration_seconds": 3.2
    }
  ],
  "total": 1
}
```

GET /api/snapshots/{filename} — ดูภาพ snapshot

ตัวอย่าง: GET /api/snapshots/20260602_143215_192_168_10_55.jpg

Response: ไฟล์ภาพ JPEG (Content-Type: image/jpeg)

endpoint นี้ไม่ต้อง login — เพื่อให้ ทำงานได้โดยตรง

5.5 สถานะระบบ

GET /api/status — Health Check (ไม่ต้อง login)

Response (200):

```
{
  "status": "online",
  "cameras_active": 2,
  "system_id": "b475ed54",
  "uptime_seconds": 3600.5
}
```

หมายเหตุเรื่อง Authentication

Endpoint ที่มีเครื่องหมาย  ต้องส่ง JWT Token ใน Header:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
```

ถ้าไม่ส่งหรือ token หมดอายุ จะได้:

```
{ "detail": "Missing or invalid Authorization header" } // 401
```

JWT Token มีอายุ 7 วัน — หลังจากนั้นต้อง login ใหม่

6. ระบบ Authentication

ขั้นตอนการ Login

1. ผู้ใช้กดปุ่ม "Sign in with Google" ใน Web App
2. @react-oauth/google เปิด popup → ผู้ใช้เลือกบัญชี Google
3. ได้ Google credential (ID Token) กลับมา
4. React ส่ง credential ไป POST /api/auth/google
5. FastAPI verify token กับ Google (ใช้ google-auth library)
6. สร้าง/อัปเดต user ใน SQLite
7. สร้าง JWT token (อายุ 7 วัน) ส่งกลับ
8. React เก็บ token ใน localStorage
9. ทุกครั้งที่เรียก API จะส่ง token ใน Authorization header

Dev Mode

ถ้าไม่ได้ตั้ง GOOGLE_CLIENT_ID (ค่าว่าง) จะเข้า Dev Mode:

- ปุ่ม  เข้าแบบ Developer Mode" จะปรากฏ
 - กดแล้วสร้าง test user อัตโนมัติ ไม่ต้องผ่าน Google
 - สะดวกสำหรับการทดสอบ
-

7. ระบบ Pairing (จับคู่อุปกรณ์)

ทำไมต้องมี Pairing?

เพื่อยืนยันว่าผู้ใช้มือถือ มีสิทธิ์เข้าถึง ระบบ detection จริงๆ

(ต้องเห็นรหัสจากจอคอมพิวเตอร์ที่รันระบบ → เป็นหลักฐานว่าเข้าถึงได้)

ขั้นตอน

1. API Server เริ่มทำงาน → สร้างรหัส 6 หลัก (เช่น 043615)
2. รหัสมีอายุ 10 นาที → แสดงในหน้าจอ desktop หรือเรียก GET /api/pair/current
3. ผู้ใช้ login ด้วย Google → เข้าหน้า /pair
4. กรอกรหัส 6 หลัก → POST /api/pair
5. Server ตรวจสอบ: ตรงไหม? หมดอายุหรือยัง?
6. ☒ สำเร็จ → บันทึกลง SQLite → is_active = 1
7. ครั้งต่อไป login → เช็ค GET /api/pair/status → is_paired = true → ข้ามไปหน้า Dashboard

ข้อมูลสำคัญ

- System ID: UUID 8 ตัวอักษร เก็บถาวรใน system_config.json — ไม่เปลี่ยนแม้รีสตาร์ท server
- Pairing Code: ตัวเลข 6 หลัก, ใช้ได้ครั้งเดียว, หมดอายุ 10 นาที
- หลายคนใช้ร่วม: user หลายคนสามารถจับคู่กับ system เดียวกันได้
- ยกเลิกจับคู่: DELETE /api/pair → ต้องกรอกรหัสใหม่

8. ระบบ Live Camera Feed

การทำงาน

```
กล้อง ONVIF → RTSP stream → PyAV decode
↓
OpenCV resize (640x360) → YOLO pose + RF classify
↓
cv2.imencode('.jpg') → base64 string → frame_buffer dict
↓
GET /api/frames → React hook useCameraFeed (polling ทุก 1.5 วิ)
```



 → แสดงภาพบนมือถือ

ประสิทธิภาพ

ค่า	คำอธิบาย
ความละเอียดภาพ	640 x 360 px
JPEG Quality	70% (ลด bandwidth)
AI Processing	ทุก 3 เฟรม (ลดภาระ CPU)
Polling Interval	1.5 วินาที
Frame Buffer	เก็บแค่เฟรมล่าสุดของแต่ละกล้อง

9. ระบบ Fall Event Logging

การทำงาน

เมื่อ FallDetector ตรวจพบการล้ม (fall_counter >= fall_trigger_frames):

1. เรียก on_fall_callback(camera_name, frame, timestamp)
2. _on_fall() ใน playvideo.py → เรียก fall_logger.log_fall_event()
3. fall_logger ตรวจสอบ cooldown (30 วินาที ต่อ IP)
4. บันทึก JPEG → fall_snapshots/{timestamp}_{ip}.jpg
5. INSERT INTO fall_events (camera_ip, camera_name, snapshot_filename, ...)
6. Web App ดูได้ที่ GET /api/fall-events
7. ภาพ snapshot ดูได้ที่ GET /api/snapshots/{filename}

Cooldown

- ทุก IP มี cooldown 30 วินาที — ป้องกันการบันทึกซ้ำจากเหตุการณ์เดียวกัน
- ถ้าตรวจพบการล้มซ้ำภายใน 30 วินาที → ข้าม ไม่บันทึก
- Thread-safe ด้วย threading.Lock()

10. สิ่งที่ต้อง Config เพิ่มเติม

10.1 Google OAuth Client ID (ถ้าต้องการใช้ Google Login จริง)

 ไม่จำเป็นถ้าใช้ Dev Mode — ระบบทำงานได้โดยไม่ต้องมี Google Client ID

ขั้นตอน:

1. ไปที่ [Google Cloud Console](#)
2. สร้าง Project ใหม่ (หรือใช้ที่มีอยู่)
3. เปิด APIs & Services → Credentials
4. กด Create Credentials → OAuth 2.0 Client ID
5. เลือก Application type: Web application
6. ใส่ Authorized JavaScript origins:
 - `http://localhost:5173` (ตอน dev)
 - `http://192.168.x.x:8000` (IP จริงของเครื่อง)
7. คัดลอก Client ID ที่ได้

ใส่ค่า:

ไฟล์ `d:\buff_p\web_app\.env`:

```
VITE_GOOGLE_CLIENT_ID=123456789-abc.apps.googleusercontent.com
```

ไฟล์ `d:\buff_p\.env`:

```
GOOGLE_CLIENT_ID=123456789-abc.apps.googleusercontent.com
```

10.2 JWT Secret (แนะนำสำหรับ production)

ไฟล์ `d:\buff_p\.env`:

```
JWT_SECRET=ใส่-string-ยาวๆ-แบบสุ่ม-ที่นี่
```

10.3 สรุปไฟล์ .env ทั้งหมด

`d:\buff_p\.env` (API Server)

```
GOOGLE_CLIENT_ID=      # ← ใส่ Google Client ID (ถ้าใช้)
JWT_SECRET=fallguard-secret # ← เปลี่ยนสำหรับ production
```



```
PYTHONIOENCODING=utf-8    # ← แก้ปัญหา emoji บน Windows
PYTHONPATH=.              # ← ให้ Python หา module ได้
```

d:\buff_p\web_app\.env (React App)

```
VITE_GOOGLE_CLIENT_ID=    # ← ใส่ Google Client ID (ถ้าใช้)
VITE_API_BASE_URL=        # ← ปกติไม่ต้องใส่ (ใช้ proxy)
```

11. วิธีรันระบบ

โหมด Development (2 terminal)

```
# Terminal 1 — API Server
uv run python -m api_server.server
# → http://localhost:8000
# Terminal 2 — React Dev Server (hot reload)
npm run dev
# → http://localhost:5173
```

เปิดบนมือถือ (WiFi เดียวกัน): <http://<ip-คอม>:5173>

โหมด Production (1 terminal)

```
# 1. Build React
npm run build
# 2. Start FastAPI (serve ทั้ง API + React)
$env:PYTHONIOENCODING='utf-8'
uv run python -m api_server.server
# → http://localhost:8000 (ทุกอย่างรวมอยู่ที่นี่)
```

เปิดบนมือถือ: <http://<ip-คอม>:8000>

รัน Detection + API ด้วยกัน

```
# Terminal 1 — Flet Detection App (สแกนกล้อง + AI)
uv run python main.py
# Terminal 2 — API Server (ให้ Web App เข้าถึงข้อมูลได้)
$env:PYTHONIOENCODING='utf-8'
uv run python -m api_server.server
```

หมายเหตุ: ตอนนี้ Flet app กับ API server ใช้ CameraManager คนละ instance
ถ้าต้องการให้ใช้ร่วมกัน ให้เรียก set_camera_manager() ใน server.py

12. ข้อจำกัดและแนวทางพัฒนาต่อ

ข้อจำกัดปัจจุบัน

ข้อจำกัด	คำอธิบาย
ต้องอยู่ WiFi เดียวกัน	มือถือต้องเชื่อมต่อ WiFi วงเดียวกับคอม (เว้นแต่ตั้ง tunnel)
Polling ไม่ใช่ real-time	ภาพอัปเดตทุก 1.5 วินาที ไม่ใช่ video stream จริงๆ
CameraManager แยก instance	Flet app กับ API server ยังไม่แชร์ CameraManager
ไม่มี push notification	ยังไม่แจ้งเตือนทันทีเมื่อตรวจพบการล้ม

แนวทางพัฒนาต่อ (ถ้าต้องการ)

ลำดับ	ฟีเจอร์	คำอธิบาย
1	แสดง Pairing Code บน Flet	เพิ่ม widget แสดงรหัส 6 หลัก + QR Code บนหน้าจอ desktop app
2	แชร์ CameraManager	รวม API server เข้ากับ Flet app ให้ใช้ frame_buffer เดียวกัน
3	WebSocket	เปลี่ยนจาก polling เป็น WebSocket สำหรับ live feed ที่ลื่นขึ้น
4	Push Notification	แจ้งเตือนผ่าน LINE Notify หรือ Web Push เมื่อตรวจพบการล้ม
5	Cloudflare Tunnel	เข้าถึงจากนอกบ้านโดยไม่ต้อง port forwarding
6	หลาย System	รองรับผู้ใช้ 1 คน pair กับหลายระบบ (หลายบ้าน)
7	ลบประวัติ	เพิ่ม endpoint ลบ fall events เก่า (auto cleanup)

 เอกสารนี้ครอบคลุมทุกสิ่งที่ทำในการพัฒนา Fall Guard Web App
สามารถใช้เป็นข้อมูลอ้างอิงสำหรับการพัฒนาต่อหรือส่งมอบงาน